Parallel Acceleration and Performance Analysis of Monte Carlo Simulation for Gold Investment Risk Assessment Using OpenMP and MPI

Andhika Narawangsa Susilo
Institut Teknologi Bandung
Bandung, Indonesia
13222036@mahasiswa.itb.ac.id

Jovan Hosea H. N.
Institut Teknologi Bandung
Bandung, Indonesia
13622005@mahasiswa.itb.ac.id

Kayla Pramudio B. A.
Institut Teknologi Bandung
Bandung, Indonesia
13222117@mahasiswa.itb.ac.id

***Abstract* – Monte Carlo simulation for financial risk analysis is computationally intensive because each stochastic price path requires multiple time-step updates. This study implements and compares OpenMP and MPI parallelization of a 252-step Geometric Brownian Motion (GBM) Monte Carlo model for gold investment risk assessment. The C++ implementations were evaluated on an Intel Core i5-12500H processor using 1, 2, 4, 8, 12, and 16 workers across workloads ranging from 10,000 to 100,000,000 simulation paths. For the primary benchmark of 1,000,000 paths, OpenMP achieved 3.18 s with an 8.88× speedup using 16 threads, while MPI achieved 3.30 s with an 8.60× speedup using 16 ranks. Scalability analysis showed near-linear speedup up to four workers, after which synchronization and communication overhead gradually limited performance. Execution time breakdown revealed that MPI overhead accounted for 64.65% of the total runtime for the 10,000-path workload at 16 workers, but decreased to only 0.02% at 100,000,000 paths as computation increasingly dominated execution. Both implementations produced terminal-price estimates with relative errors below 0.04% from the analytical GBM expectation while generating consistent probability-of-loss, VaR, and CVaR estimates. These results demonstrate that OpenMP provides the best performance on a single shared-memory node, whereas MPI becomes increasingly competitive for large computational workloads.**

Keywords—High-Performance Computing, OpenMP, MPI, Monte Carlo, Geometric Brownian Motion, Value at Risk.

I. Introduction

The primary challenge in financial quantitative computing, particularly in gold investment, lies in the inability of simple deterministic analytical formulas to comprehensively map risk distributions for real world assets characterized by stochastic dynamics and continuous market uncertainty. Calculating expected final prices alongside extreme tail-risk indicators, such as Value at Risk (VaR) for a one year horizon, cannot be sufficiently characterized by a single deterministic estimate alone.

This operational reality forces analysts to generate millions to billions of potential future market paths via Monte Carlo path generation. The computational load escalates dramatically when attempting high resolution asset pricing across a massive number of simulation paths. Processing it at a multimillion or billion iteration scale transforms the problem into a critical performance bottleneck that demands a high performance computing infrastructure to deliver rapid and actionable risk insights. The workload consists of 1,000,000 simulation paths, each discretized into 252 trading-day steps.

This study compares two parallel programming models for Monte Carlo-based gold investment risk analysis: OpenMP and MPI. OpenMP distributes independent simulation paths among multiple threads within a shared-memory environment, whereas MPI distributes simulation paths among multiple ranks with separate memory spaces. The implementations are evaluated using 1,000,000 simulation paths and 252 time steps per path on a local multicore processor.

II. Baseline and Existing Computational Approaches

The standard computational baseline currently deployed by financial analysts to address stochastic probability problems relies on executing sequential Monte Carlo simulation. High level interpreted languages such as Python (leveraging Pandas or NumPy frameworks), the R language, or scripting macros inside Microsoft Excel are heavily utilized.

This conventional approach threads simulation iterations sequentially within a single threaded execution loop on a single CPU core. While highly straightforward to implement programmatically, this method suffers from severe scalability limits and prohibitive execution times. When confronted with institutional-grade computational requirements, such as evaluating millions independent simulation trajectories, the sequential approach under interpreted runtimes frequently triggers memory exhaustion errors or demands hours of runtime, rendering it obsolete for real-time strategic risk management and dynamic decision-making.

III. Parallel Computational Design

A. Parallelization Startegy

The Monte Carlo simulation is parallelized by exploiting the independence of individual simulation paths. Each path starts from the same calibrated initial parameters but uses an independent random-number sequence to generate a different gold-price trajectory. Therefore, the total number of simulation paths can be partitioned among multiple computational workers without requiring communication during the path-generation stage.

For a total of N simulation paths and p workers, each worker is assigned approximately N/p paths. Every worker independently performs the 252-step Geometric Brownian Motion calculation for its assigned paths and accumulates local statistical results, including the terminal-price sum,

squared-price sum, loss count, drawdown, minimum price, maximum price, and terminal-price histogram.

After all workers finish their assigned paths, the local statistics are aggregated into global results. The final aggregated histogram is then used to estimate percentile values, Value at Risk, and Conditional Value at Risk. This approach minimizes communication during the computationally intensive simulation phase and performs synchronization only during the final aggregation stage.

B. Unified Parallel Computational Architecture

The proposed computational architecture consists of four main stages: parameter initialization, workload partitioning, parallel path simulation, and result aggregation. During parameter initialization, the system reads the historical gold-price and risk-free-rate datasets to determine the initial price, risk-free rate, and annualized volatility. These parameters are shared or distributed to all computational workers before the simulation begins.

In the workload partitioning stage, the total Monte Carlo paths are divided among the available workers. Each worker initializes an independent random-number generator and simulates its assigned price paths using the 252-step Geometric Brownian Motion model. To avoid race conditions and reduce synchronization overhead, each worker stores terminal-price frequencies in a local histogram and maintains local statistical accumulators.

After the parallel simulation stage is completed, the local histograms and statistical accumulators are combined into a global distribution. The global results are then used to calculate the mean terminal price, standard deviation, probability of loss, drawdown, percentile values, Value at Risk, and Conditional Value at Risk.

The same logical architecture is implemented using two parallel programming models. In OpenMP, workers are implemented as threads within a shared-memory environment. In MPI, workers are implemented as ranks with separate memory spaces, where the final aggregation is performed through MPI communication primitives.

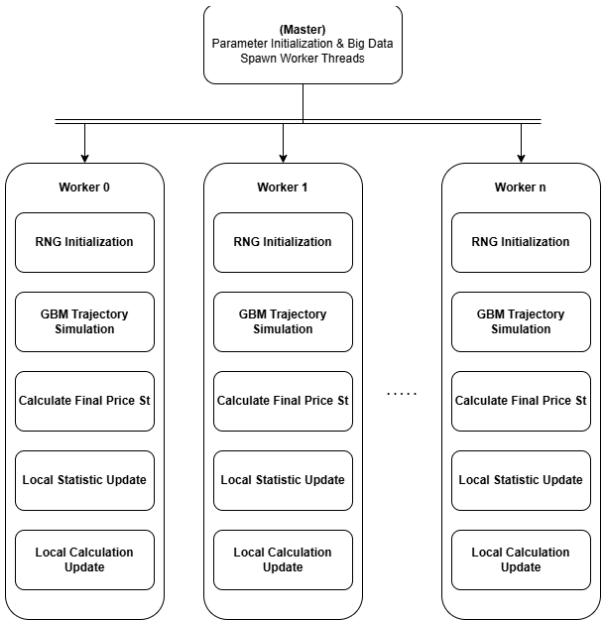

Fig 1. Parallel Worker Architecture

C. Computational Workflow

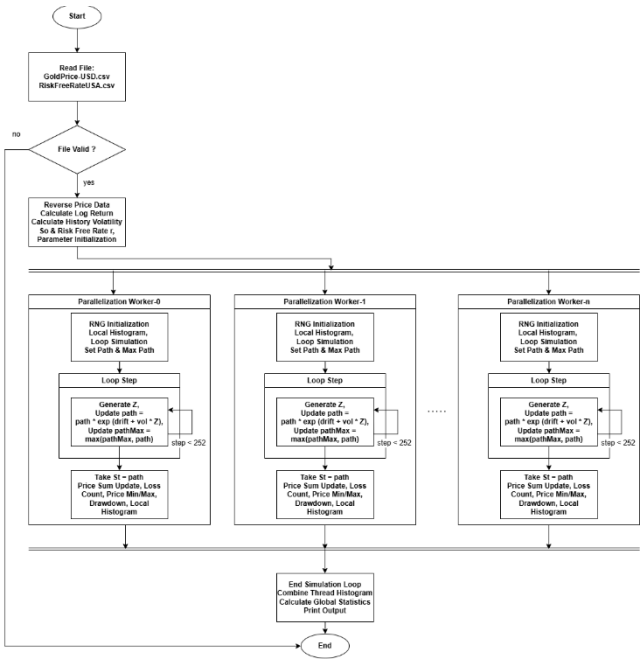

Fig 2. Monte Carlo Simulation Flowchart Diagram

The workflow is identical for both OpenMP and MPI during the Monte Carlo computation stage. The difference lies only in the execution mechanism: OpenMP assigns paths to threads through a shared-memory parallel loop, whereas MPI assigns paths to ranks and aggregates local results using message-passing operations.

- Input Phase: The system extracts raw historical parameters from the GoldPrice-USD.csv and RiskFreeRateUSA.csv datasets.
- Pre-Computation Phase: The orchestrator calculates the annualized historical volatility and the corresponding real-world risk-free rate.
- Parallel Execution Phase: Calibrated baseline variables trigger the parallel simulation loop. Each micro-thread autonomously evaluates the terminal exponential price step driven by independent standard normal random samples.
- Aggregation Phase: Frequency bin arrays are accumulated globally into the memory workspace of Rank 0 to reconstruct the terminal probability distribution.

IV. IMPLEMENTATION

The Monte Carlo risk-analysis system was implemented in ISO C++17 using two functionally equivalent parallel versions: OpenMP and MPI. The OpenMP implementation uses shared-memory parallelism, where simulation paths are distributed among multiple threads within a single process. The MPI implementation uses distributed-memory parallelism, where simulation paths are distributed among multiple ranks. Both implementations were compiled using optimization-enabled GCC-based toolchains with the -O3 optimization flag.

The input module reads historical gold-price and risk-free-rate data from semicolon-separated CSV files. Comma characters contained in numerical values are removed before the data are converted into floating-point values. The historical gold-price data are then used to calculate annualized volatility from daily logarithmic returns, while the latest gold price and risk-free rate are used as the initial GBM parameters.

Each Monte Carlo path represents one year of gold-price movement divided into 252 trading-day intervals. Therefore, the simulation applies the discretized Geometric Brownian Motion update iteratively for each time step:

$$S_T = S_0 \exp(\left(r - \frac{\sigma^2}{2}\right)\Delta t + \sigma\sqrt{\Delta t}Z)$$

where (St) is the gold price at time (t), (r) is the risk-free rate used as the drift parameter, (sigma) is the annualized historical volatility, (Zt) is a standard normal random variable, and (Delta t = 1/252). Each simulation path is updated 252 times before its final terminal price is obtained.

Each computational worker initializes an independent 64-bit Mersenne Twister random-number generator, {std::mt19937_64}, with a worker-specific seed. In the OpenMP implementation, each thread maintains its own random-number generator and local histogram. In the MPI implementation, each rank maintains its own random-number generator and local histogram. This design avoids race conditions during random-number generation and histogram updates.

To reduce memory usage, the implementation does not store all terminal prices individually. Instead, each worker maps its terminal-price results into a fixed 10,000-bin local histogram. After the simulation stage is completed, the local histograms and statistical accumulators are combined into global results. In OpenMP, this aggregation is performed after the parallel region finishes, whereas in MPI, the local results are combined using reduction operations. The global histogram is then used to estimate percentile values, Value at Risk, and Conditional Value at Risk. Because the histogram uses fixed bins, the resulting percentile-based risk metrics are treated as efficient approximations of the terminal-price distribution.

V. EXPERIMENTAL RESULTS AND ANALYSIS

1. Experimental Configuration

Table I. Experimental Configuration

Parameter	Configuration
Processor	Intel Core i5-12500H, 12th Generation
Physical cores	12 cores
Logical processors	16 logical processors
Base clock	2.50 GHz
Parallel models	OpenMP and MPI
Worker configurations	1, 2, 4, 8, 12, and 16
Simulation paths	1,000,000
Time steps per path	252
Historical gold-price records	2,189 observations
Initial price (S0)	$4,482.32
Risk-free rate (r)	4.47%
Historical volatility	17.63%
Programming language	C++
OpenMP compilation	g++ -O3 -std=c++17 -fopenmp
MPI compilation	mpicxx -O3 -std=c++17
Operating system	Windows 11 Home Single Language 25H2
Memory	16GB

2. Benchmark Timing Scope

To ensure a consistent interpretation of the benchmark results, the reported execution time was defined as the elapsed time of the parallel execution phase. Table II summarizes the components included and excluded from the measured execution time for the OpenMP and MPI implementations. The measurement includes the major parallel computation and synchronization costs, while file reading, GBM parameter calibration, final risk-metric calculation, and terminal output are excluded.

Table II. Benchmark Timing Scope

Component	OpenMP	MPI
MPI runtime initialization (MPI_Init)	–	Included
Thread creation / entry into parallel region	Included	–
Workload partitioning	Included	Included
Worker-specific RNG seed initialization	Included	Included
Initial synchronization	–	Included
Monte Carlo path simulation	Included	Included
Random-number generation	Included	Included
Local statistics and histogram updates	Included	Included
Thread/process synchronization	Included	Included
Statistical reduction	Included	Included
Histogram aggregation	Included	Included
MPI inter-process communication	–	Included
CSV file reading and GBM calibration	Excluded	Excluded
Final VaR/CVaR calculation and output printing	Excluded	Excluded
OS-level process creation by mpiexec before main()	–	Excluded

3. Computational Performance Comparison Analysis

The performance benchmark was conducted using 1,000,000 Monte Carlo simulation paths, with each path simulated over 252 Geometric Brownian Motion time steps. Both OpenMP and MPI implementations were evaluated using 1, 2, 4, 8, 12, and 16 workers under the same hardware and input-data conditions. The execution time, speedup, and parallel efficiency for each configuration are summarized in Table II.

The speedup and parallel efficiency calculated for each parallel framework is calculated using:

$$S_p = \frac{T_s}{T_p} \text{ and } E_p = \frac{S_p}{p} x\ 100\%$$

With:

$$S_p : Speedup\ using\ p\ workers$$
$$T_s : Sequential\ execution\ time$$
$$T_p : Execution\ Time\ with\ p\ workers$$
$$E_p : Effeiciency\ of\ p\ workers$$
$$p : workers\ used$$

Table III. Experimental Results on 1,000,000 Workload

Workers	Time		Speedup		Efficiency	
	OpenMP	MPI	OpenMP	MPI	OpenMP	MPI
1	28.22	28.36	1.00	1.00	100.00%	100.00%
2	13.45	13.43	2.10	2.11	104.87%	105.60%
4	6.78	7.40	4.16	3.84	104.01%	95.88%
8	4.88	5.44	5.78	5.21	72.30%	65.14%
12	3.68	3.52	7.66	8.05	63.86%	67.10%
16	3.18	3.30	8.88	8.60	55.47%	53.76%

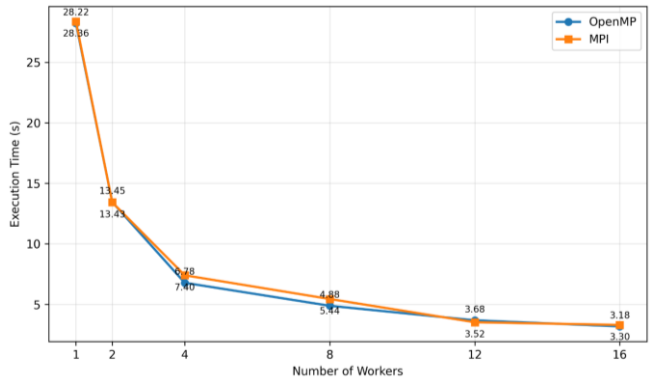

Fig 3. Execution Time vs Number of Workers

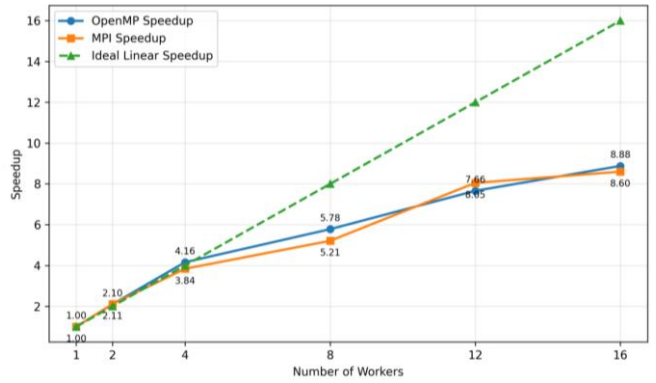

Fig 4. Speedup vs Number of Workers

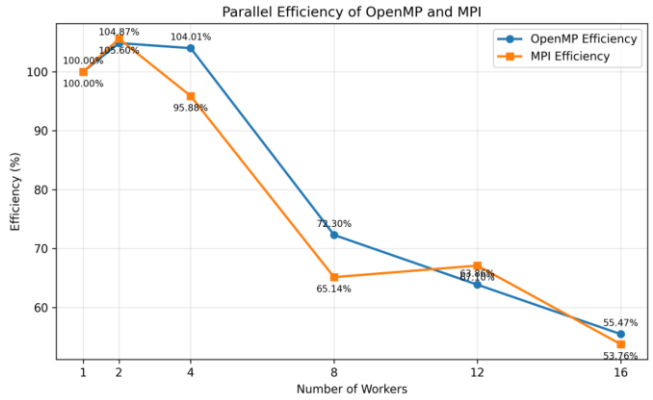

Fig 5. Parallel Efficiency vs Number of Workers

The OpenMP implementation reduced the execution time from 28.22 s using one thread to 3.18 s using 16 threads, corresponding to a maximum speedup of 8.88×. Similarly, the MPI implementation reduced the execution time from 28.36 s using one rank to 3.30 s using 16 ranks, achieving a maximum speedup of 8.60×. Both implementations demonstrate that Monte Carlo simulation is highly suitable for parallel execution because each simulation path can be calculated independently before the final aggregation stage.

As shown in Fig. 4 and Fig. 5, both OpenMP and MPI exhibit near-linear scaling at low worker counts. OpenMP achieved speedups of 2.10× and 4.16× using two and four threads, respectively, while MPI achieved speedups of 2.11× and 3.84× using two and four ranks. The slightly above-ideal efficiency values at low worker counts are likely caused by cache effects, CPU turbo boost behavior, and runtime measurement variation rather than a true superlinear parallel performance improvement.

The performance gain becomes less linear at higher worker counts. At 16 workers, OpenMP achieved the lowest execution time of 3.18 s, while MPI required 3.30 s. The parallel efficiency decreased to 55.47% for OpenMP and 53.76% for MPI at 16 workers, as illustrated in Fig. 5. This reduction is caused by synchronization overhead, memory contention, operating-system scheduling, and increased competition for processor resources.

4. Workload-Size Scalability Analysis

To evaluate the influence of computational workload on parallel scalability, additional experiments were conducted using 10,000, 100,000, 10,000,000, and 100,000,000 simulation paths, while the 1,000,000-path experiment was retained as the primary benchmark.

Table IV. Experimental Results on 10,000 Workload

Workers	Time		Speedup		Efficiency	
	OpenMP	MPI	OpenMP	MPI	OpenMP	MPI
1	0.287	0.293	1.00	1.00	100.00%	100.00%
2	0.146	0.154	1.97	1.90	98.29%	95.13%
4	0.080	0.098	3.59	2.99	89.69%	74.74%
8	0.080	0.091	3.59	3.22	44.84%	40.25%
12	0.056	0.085	5.13	3.45	42.71%	28.73%
16	0.045	0.110	6.38	2.66	39.86%	16.65%

Table V. Experimental Results on 100,000 Workload

Workers	Time		Speedup		Efficiency	
	OpenMP	MPI	OpenMP	MPI	OpenMP	MPI
1	2.809	2.834	1.00	1.00	100.00%	100.00%
2	1.374	1.357	2.04	2.09	102.22%	104.42%
4	0.679	0.720	4.14	3.94	103.42%	98.40%
8	0.515	0.565	5.45	5.02	68.18%	62.70%
12	0.404	0.425	6.95	6.67	57.94%	55.57%
16	0.334	0.394	8.41	7.19	52.56%	44.96%

Table VI. Experimental Results on 10,000,000 Workload

Workers	Time		Speedup		Efficiency	
	OpenMP	MPI	OpenMP	MPI	OpenMP	MPI
1	283.516	283.519	1.00	1.00	100.00%	100.00%
2	134.355	134.485	2.11	2.11	105.51%	105.41%
4	67.423	67.393	4.21	4.21	105.13%	105.17%
8	47.676	48.257	5.95	5.88	74.33%	73.44%
12	35.419	36.038	8.00	7.87	66.71%	65.56%
16	31.777	32.044	8.92	8.85	55.76%	55.30%

Table VII. Experimental Results on 100,000,000 Workload

Workers	Time		Speedup		Efficiency	
	OpenMP	MPI	OpenMP	MPI	OpenMP	MPI
1	2807.048	2858.53	1.00	1.00	100.00%	100.00%
2	1340.987	1343.02	2.09	2.13	104.66%	106.42%
4	671.245	670.642	4.18	4.26	104.55%	106.56%
8	478.440	472.073	5.87	6.06	73.34%	75.69%
12	338.832	345.280	8.28	8.28	69.04%	68.99%
16	314.620	279.020	8.92	10.24	55.76%	64.03%

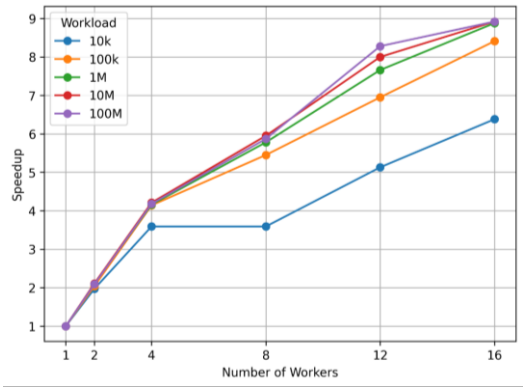

Fig 6. Speedup Across Workload Sizes on OpenMP

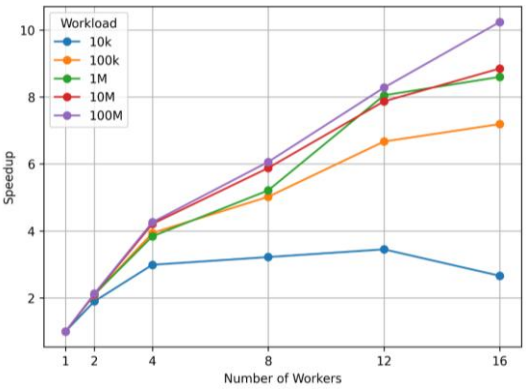

Fig 7. Speedup Across Workload Sizes on MPI

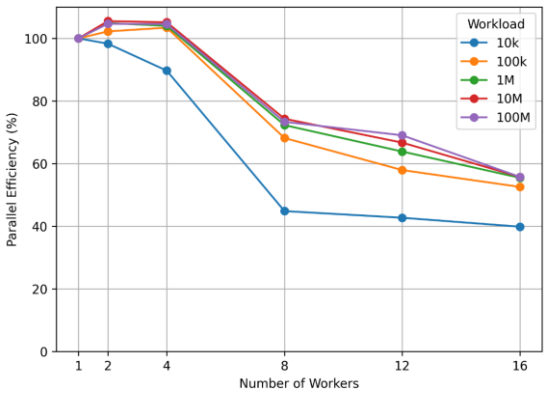

Fig 8. Parallel Efficiency Across Workload Sizes on OpenMP

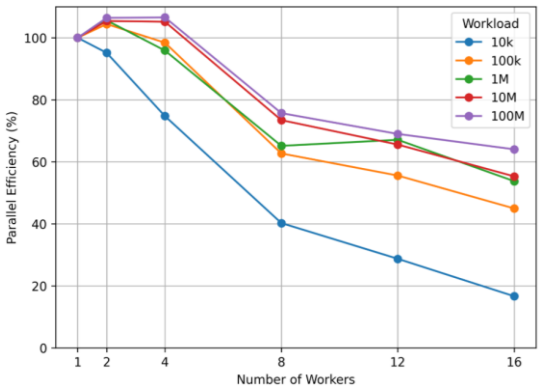

Fig 9. Parallel Efficiency Across Workload Sizes on MPI

Tables IV–VII show that execution time scales proportionally with workload size for both OpenMP and MPI. At 10,000 paths, OpenMP completes the simulation in 0.045 s while MPI requires 0.110 s using 16 workers, indicating that synchronization and reduction overhead dominate small workloads. As the workload increases, the relative overhead decreases because each worker performs substantially more independent Monte Carlo paths. Consequently, the performance gap narrows, and at 100,000,000 paths MPI outperforms OpenMP (279.020 s versus 314.620 s) at 16 workers.

Figures 6 and 7 show near-linear speedup up to four workers across all workload sizes. Beyond this point, the speedup increases more gradually due to synchronization overhead and resource contention. This effect is most evident at 10,000 paths, where OpenMP reaches only 6.38× speedup and MPI drops to 2.66× at 16 workers. In contrast, workloads of one million paths and above maintain stable scalability, with OpenMP achieving approximately 8.9× speedup and MPI reaching 10.24× at 100,000,000 paths.

These scaling trends are consistent with classical parallel performance models. The gradual speedup saturation beyond four workers follows Amdahl's Law, where the serial fraction and synchronization overhead limit the achievable speedup. Conversely, the improved scalability observed for larger workloads supports Gustafson's Law, demonstrating that increasing computational intensity reduces the relative impact of fixed parallel overhead and improves parallel efficiency.

Figures 8 and 9 show a corresponding decline in parallel efficiency as the number of workers increases. For workloads of 100,000 paths and above, both implementations maintain efficiencies close to 100% up to four workers before gradually decreasing. At 16 workers, OpenMP maintains 52.56–55.76%

efficiency, whereas MPI improves from 44.96% at 100,000 paths to 64.03% at 100,000,000 paths. Conversely, MPI efficiency falls to only 16.65% for the 10,000-path workload, indicating that parallel overhead becomes increasingly significant for small workloads.

Table VIII. Execution Time Breakdown Across Different Workload Sizes and Worker Configurations

Workload	Worker	Parallelization	Init	Compute	Sync/Overhead
10k	1	OpenMP	3.6666%	96.3334%	0.0000%
		MPI	2.4981%	91.6386%	5.8633%
	4	OpenMP	7.6924%	91.0254%	1.2823%
		MPI	5.0279%	69.5425%	25.4296%
	16	OpenMP	6.5780%	85.5260%	7.8960%
		MPI	5.1368%	30.2127%	64.6505%
1M	1	OpenMP	0.0382%	99.9583%	0.0035%
		MPI	0.0282%	99.9160%	0.0558%
	4	OpenMP	0.1481%	99.8371%	0.0148%
		MPI	0.0841%	99.6891%	0.2268%
	16	OpenMP	0.3918%	99.5178%	0.0904%
		MPI	0.1674%	97.5518%	2.2807%
100M	1	OpenMP	0.0004%	99.9996%	0.0000%
		MPI	0.0002%	99.9993%	0.0005%
	4	OpenMP	0.0015%	99.9982%	0.0003%
		MPI	0.0007%	99.9957%	0.0036%
	16	OpenMP	0.0020%	99.9970%	0.0010%
		MPI	0.0019%	99.9749%	0.0231%

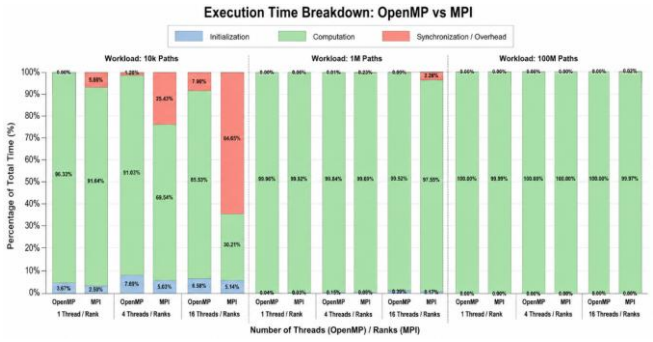

Fig 10. Execution Time Breakdown Across Different Workload Sizes and Worker Configurations

To investigate the evolution of parallel overhead, the breakdown analysis was performed using 1, 4, and 16 workers, representing sequential execution (without parallel overhead), near-ideal parallel efficiency, and the maximum parallelism where synchronization and communication overhead become most significant, respectively.

Figures 10 and Tables VIII–IX explain the scalability behavior observed in Figs. 6–9. At 10,000 paths, synchronization overhead increases rapidly with the number of workers, reaching 7.90% in OpenMP and 64.65% in MPI at 16 workers. Consequently, the computation phase occupies only 30.21% of the MPI execution time, explaining its significant degradation in speedup and parallel efficiency. In contrast, OpenMP maintains substantially lower synchronization overhead due to its shared-memory execution model.

As the workload increases to 1,000,000 and 100,000,000 paths, computation progressively dominates the execution time. At 16 workers, synchronization overhead remains below 0.1% for OpenMP and decreases from 2.28% to only 0.02% for MPI, indicating that communication costs are effectively amortized by the increasing computational workload. Consequently, both implementations exhibit comparable scalability for large workloads, with performance primarily determined by the computation-to-overhead ratio rather than the parallelization model itself.

5. Numerical Verification and Risk Metrics

a) Numerical Verification

The numerical validity of the simulation was evaluated by comparing the simulated mean terminal price with the analytical expectation of the Geometric Brownian Motion model:

$$E[S_T] = S_0 e^{(rT)}$$

Using the calibrated parameters, the analytical expected terminal price was $4687.23. The fastest OpenMP configuration, using 16 threads, produced a simulated mean terminal price of $4687.61, corresponding to a relative error of approximately 0.008%. The fastest MPI configuration, using 16 ranks, produced a simulated mean terminal price of $4685.39, corresponding to a relative error of approximately 0.039%.

These small deviations indicate that both parallel implementations preserve the expected statistical behavior of the GBM model. The remaining difference from the analytical value is expected because Monte Carlo simulation uses finite random samples and different worker configurations generate independent random-number streams.

b) Risk Metric Estimation

The terminal-price distribution was used to estimate the probability of loss, Value at Risk, and Conditional Value at Risk. The probability of loss is defined as the proportion of simulation paths with a terminal price below the initial gold price (S_0). VaR represents the estimated loss threshold at a specified confidence level, while CVaR represents the average loss within the worst tail region beyond the VaR threshold.

Table IX summarizes the numerical verification and risk metrics obtained from the fastest OpenMP and MPI configurations. Both implementations produced closely comparable risk estimates, with small differences caused by stochastic sampling variation and the use of independent random-number sequences.

Table IX. Numerical Verification and Risk Metrics of the Parallel Configurations

Metric	OpenMP (16 Threads)	MPI (16 Ranks)
Execution Time (s)	3.18	3.297
Simulated Mean Price	$4,687.61	$4,685.39
Analytical Mean Price	$4,687.23	$4,687.23

Relative Error	0.01%	0.04%
Final Price Std Dev	$832.44	$832.68
Probability of Loss	43.24%	43.52%
Average Drawdown	10.91%	10.93%
VaR 95%	$1,029.08	$1,030.57
VaR 99%	$1,419.46	$1,422.44
CVaR 95%	$1,267.71	$1,269.74

The results indicate that the estimated probability of loss is approximately 43%, meaning that a substantial portion of the simulated one-year scenarios end below the initial gold price. At the 95% confidence level, the estimated VaR is approximately $1030 per unit of gold-price exposure, while the CVaR indicates that the average loss within the worst 5% of scenarios is approximately $1268–$1270. Because the percentile values are derived from a fixed-bin histogram, the reported VaR and CVaR values should be interpreted as efficient approximations of the simulated terminal-price distribution.

The source code and the result for the simulation is available here

VI. Conclusion

This study evaluated OpenMP and MPI implementations of a 252-step GBM Monte Carlo simulation for gold investment risk assessment using workloads ranging from 10,000 to 100,000,000 simulation paths. Both implementations exhibited near-linear speedup up to four workers, while performance gradually saturated at higher worker counts due to synchronization and communication overhead. At the primary benchmark of 1,000,000 paths, OpenMP achieved 3.18 s with an 8.88× speedup, whereas MPI achieved 3.30 s with an 8.60× speedup using 16 workers. For the largest workload (100,000,000 paths), MPI outperformed OpenMP, achieving a 10.24× speedup compared with 8.92×.

The workload-size and execution-time breakdown analyses showed that scalability is primarily governed by the computation-to-overhead ratio. Small workloads are dominated by synchronization and communication overhead, particularly for MPI, where overhead reached 64.65% of the total execution time at 10,000 paths using 16 workers. As the workload increased, computation accounted for more than 99.9% of the total execution time, effectively amortizing the parallel overhead and enabling both implementations to achieve comparable scalability.

Both implementations preserved the expected statistical behavior of the GBM model, with mean terminal-price errors below 0.04% relative to the analytical expectation. Overall, OpenMP provides the best performance for single-node shared-memory systems, whereas MPI becomes increasingly competitive as computational intensity grows and offers a scalable foundation for future multi-node distributed execution.

References

[1] N. A. Zakaria, M. F. Ahmad, and M. S. M. Kasihmuddin, “Modeling and forecasting gold price using Geometric Brownian Motion,” AIP Conf. Proc., vol. 2138, no. 1, pp. 030018-1–030018-6, Aug. 2019.

[2] S. Maruddani and A. Prahutama, “Value at Risk portfolio using Monte Carlo simulation,” J. Phys.: Conf. Ser., vol. 855, no. 1, pp. 012027-1–012027-7, May 2017.

[3] V. Alexandrov, “Parallel Monte Carlo algorithms for financial derivatives pricing,” Math. Comput. Simul., vol. 47, no. 2, pp. 113–122, Aug. 1998.

[4] M. B. Giles, “Multi-level Monte Carlo path simulation,” Oper. Res., vol. 56, no. 3, pp. 607–617, Jun. 2008.

[5] T. J. Linsmeier and N. D. Pearson, “Value at Risk,” Financ. Anal. J., vol. 56, no. 2, pp. 47–67, Mar. 2000.

[6] J. L. Gustafson, "Amdahl's Law," in *Encyclopedia of Parallel Computing*, D. Padua, Ed. Boston, MA, USA: Springer, 2011, pp. 53–60.

[7] P. Pacheco, *An Introduction to Parallel Programming*. Burlington, MA, USA: Morgan Kaufmann, 2011.